DOCUMENTO 03

Arquitectura y portabilidad

Estructura del sistema para preservar las reglas de negocio, aislar la infraestructura y planear un cambio de almacenamiento sin falsos supuestos.

DIRECCIÓN

Web → API → Core → Ports → SQLite/Drizzle: las dependencias apuntan hacia adentro.

AUTORIDAD

El Core decide; la base de datos protege los invariantes físicos.

BENEFICIO

Cambiar el almacenamiento no debe reescribir las reglas operativas.

LÍMITE

NoSQL no es un reemplazo mecánico: exige modelo y consultas propios.

Seis workspaces

Cada workspace tiene una responsabilidad acotada y dependencias declaradas.

`packages/core` `packages/sqlite` `packages/contracts` `packages/config` `apps/api` `apps/web`

Arquitectura en capas

La Web presenta la operación; la API expone la entrada; el Core aplica casos de uso y políticas; los Ports definen contratos; SQLite/Drizzle implementa la persistencia.

Web

`apps/web` es una interfaz Next.js que llega a la API solo por HTTP y nunca importa el core ni SQLite.

API

`apps/api` sirve Hono REST bajo `/v1` con `@hono/node-server`, dividido en interface, infrastructure y composition.

Core

Decide reglas, estados y transiciones, permisos y cierres. No tiene dependencias en tiempo de ejecución.

Ports

Aíslan los contratos de repositorios y transacciones entre las decisiones del core y el motor de datos.

SQLite / Drizzle

Implementa los Ports sobre `node:sqlite`; las migraciones son SQL escrito a mano y Drizzle es solo la capa de consulta tipada.

Settings

Configura las políticas sin convertir la interfaz ni la base de datos en fuente de verdad.

INVARIANTES VERIFICADOS POR PRUEBAS

- El core no importa adaptadores ni frameworks HTTP.
- Los adaptadores nunca llaman a los casos de uso.
- `hono` se importa solo bajo `apps/api/src/interface/http/`.
- La capa HTTP nunca importa la base de datos, la infraestructura ni la composición.
- Solo la raíz de composición construye adaptadores SQLite.
- `apps/web` nunca importa `@opskit/core` ni `@opskit/sqlite`.
- `drizzle-orm` solo lo importa `packages/sqlite`.

VERIFICACIÓN REAL

Estas reglas las comprueban pruebas que analizan el árbol de producción y además prueban sus propios detectores, de modo que no pueden dejar de detectar en silencio.

Quién decide y quién protege

La decisión y la protección viven en capas distintas, y el borde solo transporta la solicitud.

CAPA	RESPONSABILIDAD	EJEMPLO
Core	Decisión de negocio	Solo un período `calculated` puede cerrarse.
Port	Contrato intercambiable	Repositorio de facturas o de caja.
SQLite / Drizzle	Persistencia y protección	Restricciones, claves foráneas, índices y triggers.
Web y API	Experiencia y borde	Validar la solicitud y presentar el error.

REGLA DE DISEÑO

La base de datos no reemplaza los casos de uso. Sus restricciones son un respaldo físico; el Core conserva la decisión y el orden de las transiciones.

Portabilidad real

Cuatro reglas que sostienen un cambio de motor sin degradarlo.

No copiar SQL

Un motor nuevo requiere un adaptador que implemente los Ports, no SQL trasladado a ciegas.

No omitir migraciones

El cambio debe incluir migración, validación de datos y un plan de reversa.

No ignorar concurrencia

Aislamiento, bloqueos, transacciones e idempotencia cambian de un motor a otro.

NoSQL con modelo propio

Documentos, claves y patrones de consulta se diseñan según el acceso, no como tablas disfrazadas.

Cambio de base de datos: plan mínimo

Cinco frentes que deben resolverse juntos; ninguno sustituye a los otros.

FRENTE	ACCIÓN CONCRETA	RIESGO QUE REDUCE
Contrato	Implementar el adaptador de Ports	Dependencia del motor dentro del Core.
Datos	Diseñar y ejecutar migraciones verificables	Pérdida de datos, tipos inconsistentes o saldos erróneos.
Concurrencia	Revisar unidad de trabajo, transacciones, bloqueo e idempotencia	Doble cobro, cierre simultáneo o condiciones de carrera.
Consultas	Medir índices y patrones de lectura y escritura	Regresión de rendimiento.
Operación	Respaldos, observabilidad y reversa	Recuperación incompleta ante un fallo.

Migraciones actuales	De 001 a 008, en SQL escrito a mano; Drizzle nunca las genera ni las aplica.
Integridad	Esquema, claves foráneas, checks, índices y triggers escritos a mano son la autoridad física.
Apertura	La base abre con `journal_mode=WAL` y `foreign_keys=ON`.

CHECKSUMS DE MIGRACIÓN

Cada migración tiene contenido canónico explícito y su checksum se guarda como `v2:<sha256>`. Un valor `v2:` que no coincide siempre falla; nunca se sobrescribe en silencio.

CRITERIO DE SALIDA

El cambio es aceptable cuando conserva las reglas del Core, pasa las migraciones y las pruebas de concurrencia, y demuestra que caja, facturación, inventario y compensación mantienen su trazabilidad.